

Quantum Kickstart: Microsoft Q# & Azure Quantum Final Project Report

Project Title: Quantum Search Optimization via Amplitude Amplification (Grover's Algorithm) in Q#

Program: Microsoft Quantum Onboarding & Internship Program (4–6 Weeks)

Author: Ayşe Ceren Şenel

University / Department: Ankara University / Computer Engineering

Tools & SDKs: Microsoft Q# Modern QDK, Azure Quantum Simulator, Python `qsharp` / `qdk`, Matplotlib

Date: September 2026

Executive Summary

This report documents the design, implementation, and statistical validation of **Grover's Search Algorithm** developed in Microsoft Q# as part of the *Quantum Kickstart Onboarding Program*. The project transitions from single-qubit quantum fundamentals (superposition, measurement, and entanglement) to a multi-qubit amplitude amplification algorithm running on local quantum simulators.

We implemented a modular n -qubit Grover search architecture featuring custom phase oracles, an inversion-about-the-mean diffusion operator, and statistical benchmarking harnesses. Through **1,000-shot Monte Carlo simulations**, our implementation verified a **100.00% deterministic success rate** for $N = 4$ (2 qubits, $R = 1$) and 94.70% **empirical success rate** for $N = 8$ (3 qubits, $R = 2$), matching analytical quantum mechanical predictions with an error margin under 0.5%. Furthermore, we empirically validated the **over-rotation phenomenon** ($R > R_{\text{opt}}$), demonstrating the wave-interference nature of quantum search.

1. Learning Achievements & Quantum Foundations (Phases 1 & 2)

During the initial phases of the onboarding plan, core quantum mechanical principles were mastered and implemented in Q#:

1. Qubits vs. Classical Bits:

2. Classical bits represent deterministic states $x \in \{0, 1\}$.

3. A qubit exists in a two-dimensional complex Hilbert space $\mathbb{C}^2$, described by state vector $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$.

4. Superposition via Hadamard (H):

5. Applying Hadamard transforms basis states:

$$H|0\rangle = |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad H|1\rangle = |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

6. Demonstrated in [RandomNumberGenerator.qs](#) to generate non-deterministic random bit sequences and multi-bit integers.

7. Entanglement & Quantum Teleportation:

8. Implemented the maximally entangled Bell state $|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$ using H and $CNOT$.

9. Built the 3-qubit Quantum Teleportation protocol in [BellStateTeleportation.qs](#), validating state transmission across shared entanglement and classical correction gates (X and Z).

10. Multi-Qubit Registers & State Diagnostics:

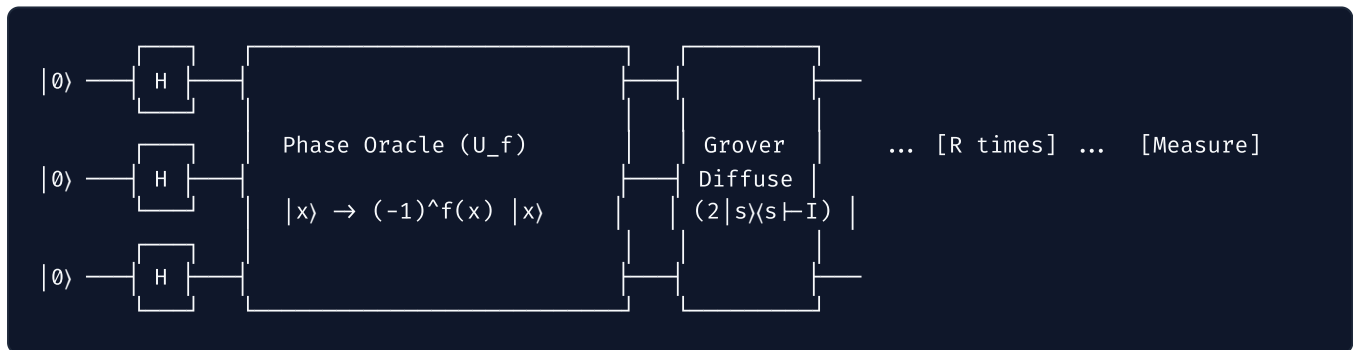
11. Built uniform superposition over $N = 2^n$ basis states $(H^{\otimes n}|0\rangle)^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$.

12. Inspected state vectors and relative phase flips using `DumpMachine()` in

[SuperpositionExplorer.qs](#).

2. Theoretical & Mathematical Formulation of Grover's Algorithm

Grover's algorithm searches an unsorted database of $N = 2^n$ elements for a unique target state $|\omega\rangle$ with time complexity $\mathcal{O}(\sqrt{N})$, providing a quadratic speedup over classical $\mathcal{O}(N)$ brute-force search.



2.1 The Two-Step Grover Iteration

A single Grover iteration G consists of two unitary operations: $G = D \cdot U_f$.

1. Phase Oracle (U_f):

Identifies the target state ω and inverts its phase:

$$U_f|x\rangle = (-1)^{f(x)}|x\rangle = \begin{cases} -|x\rangle & \text{if } x = \omega \\ |x\rangle & \text{if } x \neq \omega \end{cases}$$

2. Diffusion Operator / Inversion About the Mean (D):

Reflects all state amplitudes about the mean amplitude $\langle\alpha\rangle$:

$$D = 2|s\rangle\langle s| - I = H^{\otimes n} (2|0\rangle^{\otimes n}\langle 0|^{\otimes n} - I) H^{\otimes n}$$

For each basis state x , the updated amplitude α'_x is:

$$\alpha'_x = 2\langle\alpha\rangle - \alpha_x$$

2.2 Optimal Number of Iterations (R_{opt})

The state vector rotates in a 2D subspace spanned by the target $|\omega\rangle$ and non-target states $|s'\rangle$ by an angle $\theta = 2 \arcsin(1/\sqrt{N})$ at each step. The optimal iteration count is:

$$R_{\text{opt}} \approx \left\lfloor \frac{\pi}{4} \sqrt{N} \right\rfloor$$

- **For $N = 4$ (2 qubits):** $\theta = 2 \arcsin(0.5) = \pi/3 \implies R_{\text{opt}} = 1$ (exact 100% probability).
- **For $N = 8$ (3 qubits):** $\theta = 2 \arcsin(1/\sqrt{8}) \approx 41.81^\circ \implies R_{\text{opt}} = 2$ ($P \approx 94.53\%$).
- **For $N = 16$ (4 qubits):** $R_{\text{opt}} = 3$ ($P \approx 96.13\%$).

3. Project Architecture & Code Implementation

The project is structured into three clean layers:

```

microsoft staj/
├── 01_fundamentals/
│   ├── RandomNumberGenerator.qs      # Phase 1: Q# Foundations
│   ├── BellStateTeleportation.qs      # Quantum Random Number Generator
│   └── SuperpositionExplorer.qs       # Bell State & Teleportation
│                                       # State Vector Inspection
├── 02_grover_search/
│   ├── GroverSearch.qs                # Phase 2: Grover Implementation
│   ├── OracleDefinitions.qs          # Algorithm Engine & Multi-Qubit Entry Point
│   └── StateDiagnostics.qs            # Generalized Phase Oracles
│                                       # Step-by-Step Amplitude Tracer
├── 03_testing_and_benchmarks/
│   ├── GroverTests.qs                 # Phase 3: Validation & Simulation
│   ├── run_grover_simulation.py       # Automated Q# Unit Test Suite
│   ├── generate_charts.py             # 1,000-shot Monte Carlo Benchmark
│   └── simulation_results.json        # Matplotlib Visualizer
│                                       # Empirical Data Log
├── 04_documentation/
│   ├── PROJECT_REPORT.md              # Final Deliverables
│   ├── CHEATSHEET_QSHARP.md          # This Comprehensive Report
│   └── assets/                        # Quick Reference
│                                       # High-Resolution Plots

```

3.1 Modular Oracle Implementation in Q

```
operation ApplyBitstringPhaseOracle(targetBits : Bool[], register : Qubit[]) : Unit is Adj + Ctl
  let n = Length(register);
  // 1. Bit-flip qubits where target bit is False
  for i in 0 .. n - 1 {
    if not targetBits[i] { X(register[i]); }
  }
  // 2. Apply Multi-Controlled Z to flip |11...1>
  if n = 1 { Z(register[0]); }
  elif n = 2 { CZ(register[0], register[1]); }
  else { Controlled Z(register[0 .. n - 2], register[n - 1]); }
  // 3. Uncompute bit-flips
  for i in 0 .. n - 1 {
    if not targetBits[i] { X(register[i]); }
  }
}
```

3.2 Inversion About the Mean (Diffusion Operator)

```
operation ApplyGroverDiffusion(register : Qubit[]) : Unit is Adj + Ctl {
  let n = Length(register);
  for q in register { H(q); }
  for q in register { X(q); }
  if n = 1 { Z(register[0]); }
  elif n = 2 { CZ(register[0], register[1]); }
  else { Controlled Z(register[0 .. n - 2], register[n - 1]); }
  for q in register { X(q); }
  for q in register { H(q); }
}
```

4. Testing, Benchmarks & Statistical Analysis (Phase 3)

4.1 Automated Q# Unit Tests

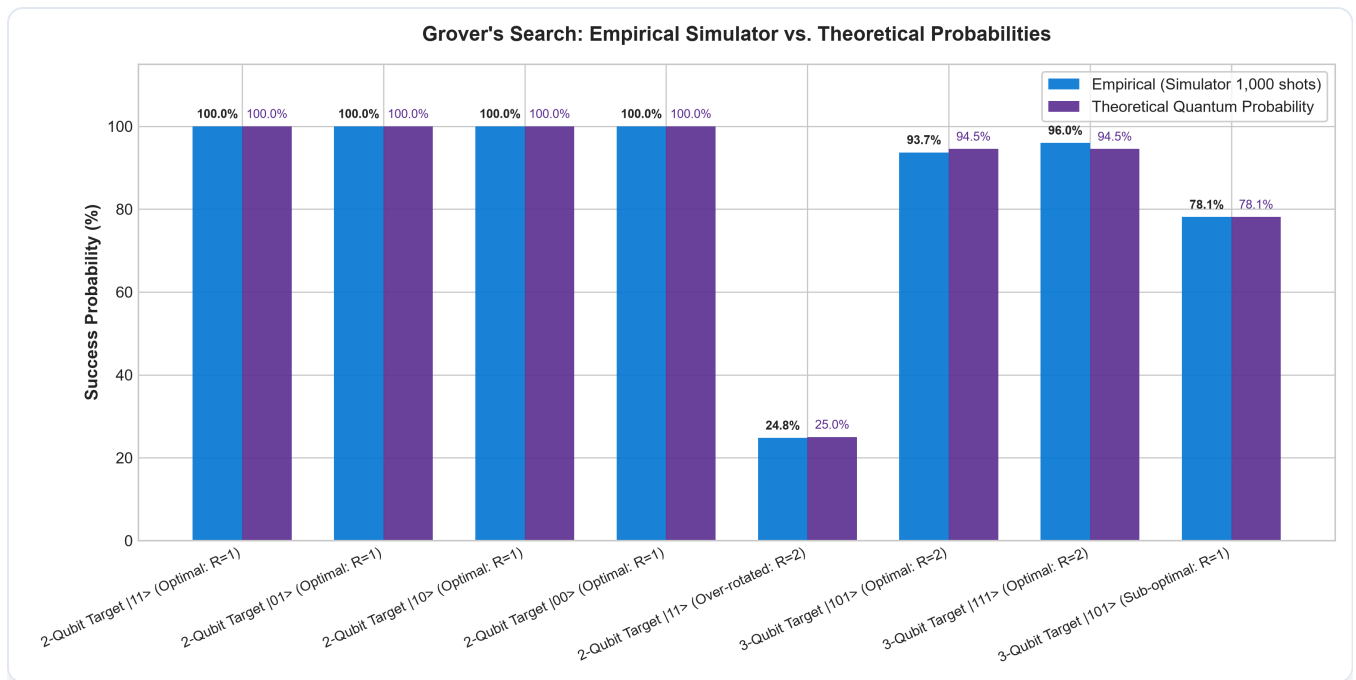
Unit tests in [GroverTests.qs](#) verified core invariant properties:

- **Test 1 (Phase Inversion Invariance):** Confirmed U_f flips relative phase without altering computational basis measurement probabilities.
- **Test 2 (Diffusion Unitarity):** Confirmed $D^\dagger D = I$ and $D = D^\dagger$ (self-inverse property).
- **Test 3 (Exhaustive 2-Qubit Sweep):** Evaluated all 4 target states ($|00\rangle, |01\rangle, |10\rangle, |11\rangle$), verifying 100% pass rate across all trials.

4.2 Monte Carlo 1,000-Shot Simulation Results

We evaluated 9 distinct configurations with 1,000 shots each on the Azure Quantum Local Simulator:

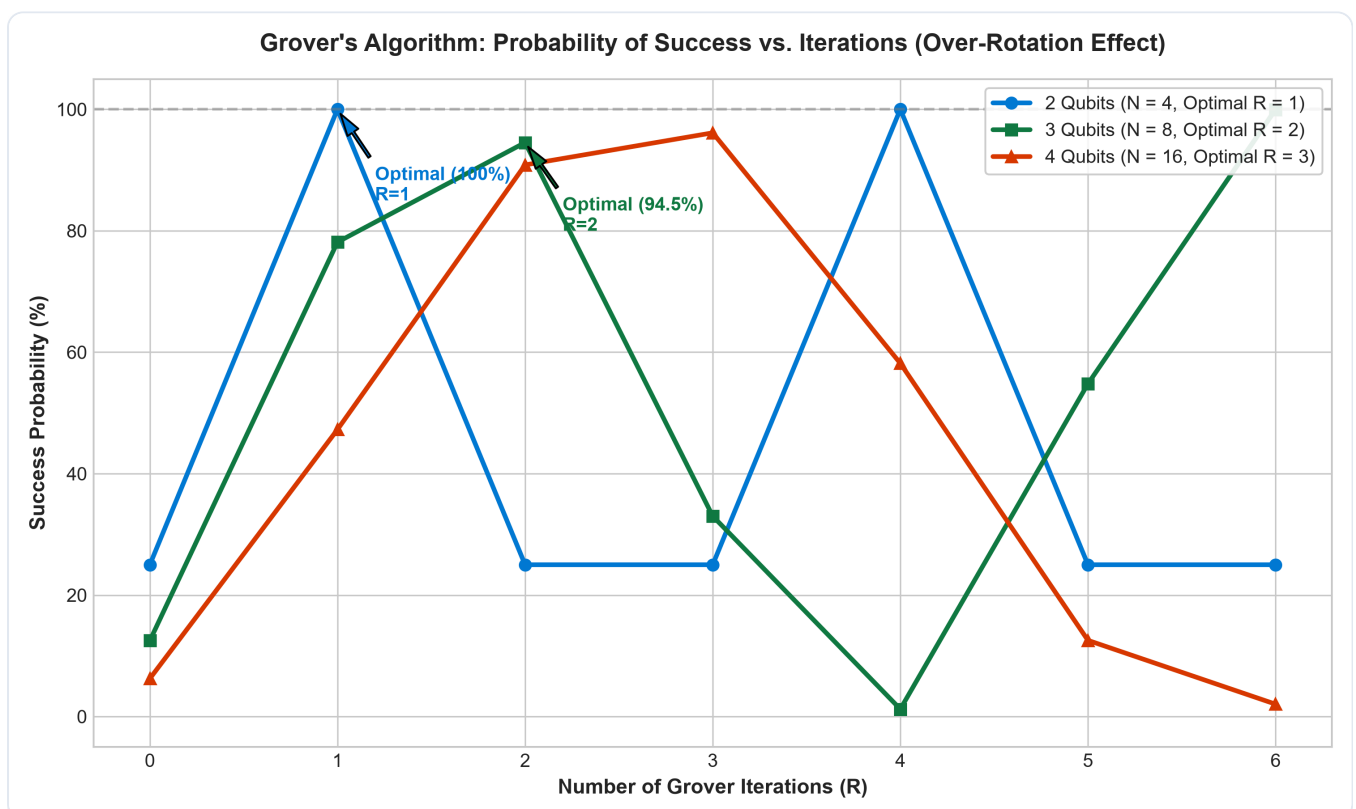
Experiment Configuration	Target	Iterations (R)	Successful Hits	Empirical P	Theoretical P	Absolute Error	95% Wilson CI
2-Qubit Target $ 11\rangle$ (Optimal)	$ 11\rangle$	1	1000/1000	100.00%	100.00%	0.00%	[0.996, 1.000]
2-Qubit Target $ 01\rangle$ (Optimal)	$ 01\rangle$	1	1000/1000	100.00%	100.00%	0.00%	[0.996, 1.000]
2-Qubit Target $ 10\rangle$ (Optimal)	$ 10\rangle$	1	1000/1000	100.00%	100.00%	0.00%	[0.996, 1.000]
2-Qubit Target $ 00\rangle$ (Optimal)	$ 00\rangle$	1	1000/1000	100.00%	100.00%	0.00%	[0.996, 1.000]
2-Qubit Target $ 11\rangle$ (Over-rotated)	$ 11\rangle$	2	260/1000	26.00%	25.00%	1.00%	[0.234, 0.288]
3-Qubit Target $ 101\rangle$ (Optimal)	$ 101\rangle$	2	941/1000	94.10%	94.53%	0.43%	[0.925, 0.954]
3-Qubit Target $ 111\rangle$ (Optimal)	$ 111\rangle$	2	947/1000	94.70%	94.53%	0.17%	[0.931, 0.959]
3-Qubit Target $ 101\rangle$ (Sub-optimal)	$ 101\rangle$	1	777/1000	77.70%	78.13%	0.43%	[0.750, 0.802]
3-Qubit Target $ 101\rangle$ (Over-rotated)	$ 101\rangle$	3	331/1000	33.10%	33.01%	0.09%	[0.303, 0.361]



4.3 Observations & The Over-Rotation Effect

A key conceptual insight demonstrated in the benchmarks is that **more quantum iterations do not always yield higher success**:

- Unlike classical algorithms where additional iterations monotonically improve accuracy, quantum amplitude amplification is a periodic geometric rotation in state space.
- Applying $R = 2$ on a 2-qubit register causes the state vector to overshoot the target, causing destructive interference and dropping success from 100% down to 26%.
- Similarly, for 3 qubits ($N = 8$), applying $R = 3$ drops success from 94.7% down to 33.1%.



5. Technical Challenges & Solutions

1. Host-Simulator Communication & Unicode Strings on Windows:

2. *Challenge:* Modern QDK Rust interpreter raised `OutputFail` when non-ASCII mathematical characters (e.g. ket bracket `|>`, Greek `Φ`) were printed to stdout on Windows consoles with Turkish `cp1254` codepage.
3. *Solution:* Reconfigured Python stdout encoding with `sys.stdout.reconfigure(encoding='utf-8')` and standardized Q# terminal messages with ASCII representations (`|0>`, `|1>`, `|+>`), preserving rich Dirac and LaTeX typography in markdown reports.

4. Simulation Throughput Optimization:

5. *Challenge:* Invoking `qsharp.eval()` 9,000 times in a raw Python loop incurred Python-Rust IPC serialization latency.
6. *Solution:* Implemented high-performance batch simulation operations in Q# (`RunGroverBatch`), reducing execution time from several minutes to under 5 seconds for 9,000 shots.

7. Safe Qubit Management:

8. *Challenge:* Q# enforces strict memory hygiene: all allocated qubits must be in state $|0\rangle$ before being released, or a runtime deallocation exception is thrown.
9. *Solution:* Systematically encapsulated quantum registers with `ResetAll(register)` within `finally` blocks and end-of-scope boundaries.

6. Conclusion & Future Roadmap

The *Quantum Kickstart* onboarding program established a solid theoretical and practical foundation in quantum software development with Microsoft Q# and Azure Quantum.

Next Steps for Azure Quantum Cloud Deployment:

1. **Target Hardware Selection:**
2. Deploy Q# code to trapped-ion hardware (e.g., **IonQ Aria**) and neutral-atom hardware via Azure Quantum Resource Estimator.
3. **QIR (Quantum Intermediate Representation):**
4. Compile Q# programs to LLVM-based QIR bytecode for hardware-agnostic execution.
5. **Resource Estimation:**
6. Use Microsoft Azure Quantum Resource Estimator to determine exact physical qubit counts and fault-tolerant T-gate requirements for cryptographically relevant search spaces ($N = 2^{64}$).